

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Project 1

Prepared by: Group 4



UNIVERSITI TEKNOLOGI MALAYSIA

PRESENTATION SLIDE

HOSTEL RESERVATION SYSTEM

Innovating Solutions

www.utm.my

Team Roles

01 MUHAMMAD AFIQ DANISH BIN MOHD HAZNI

- Design overall database structure
- Applied key constraints
- Ensure smooth integration between tables

02 DHESHIEGHAN A/L SARAVANA MOORTHY

- Data Manipulation & Query Developer (DML + DQL)

03 MUHAMMAD SYAHMI FARIS BIN RUSLI

- Query Specialist
- Developed all DQL queries
- Generated outputs and validated results
- Ensured accurate data retrieval across tables

04 PRAVINRAJ A/L SIVABATHI

- Indexing & Optimization Analyst

Introduction of the System

- Simple backend for managing student room booking and payments
- Designed for university hostels to organize: **applications, assignments and payments**



Objectives of the System

1. Manage student, room and reservation information efficiently
2. Provide accurate room availability and booking records
3. Reduce manual errors
4. Allow easy retrieval of data
5. Support backend operations used by student and staff



DDL

Database Structure

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    fname VARCHAR(50) NOT NULL,  
    lname VARCHAR(50) NOT NULL,  
    ic_number VARCHAR(20) NOT NULL UNIQUE,  
    phone VARCHAR(20) NOT NULL,  
    email VARCHAR(50) NOT NULL,  
    faculty VARCHAR(10) NOT NULL,  
    gender ENUM('Male', 'Female', 'Not Specified') DEFAULT 'Not Specified'  
);
```

- Tables created: *students*, *room_types*, *room_locations*, *rooms*, *reservations*, *payments*
- Several constraints are used to ensure data integrity and consistency

Constraints

01

PRIMARY KEY

Uniqueness per row

02

FOREIGN KEY

Relationship
between tables

03

UNIQUE

Prevent duplicates

04

NOT NULL

Required values

05

DEFAULT

When not inserted
with any values

06

CHECK

Enforce logical rules

DML

Data Insertion (Insert Statements)

#Inserting 10 data for each tables

```
INSERT INTO students (student_id, fname, lname, ic_number, phone, email, faculty, gender) VALUES
(1, 'Chew', 'Chiu Xian', '040101-01-1234', '0123456701', 'cxchew@graduate.utm.my', 'ENG', 'Male'),
(2, 'Joanne', 'Ching', '040102-02-2345', '0123456702', 'joanne-321@graduate.utm.my', 'SCI', 'Female'),
(3, 'Cheryl', 'Cheong', '040103-03-3456', '0123456703', 'cheryl322@graduate.utm.my', 'BUS', 'Female'),
(4, 'Lubna', 'Al Haani binti Radzuan', '040104-04-4567', '0123456704', 'haani1224@graduate.utm.my', 'ART', 'Female'),
```

```
INSERT INTO room_types (type, price, capacity) VALUES
('Single', 150.00, 1),
('Double', 250.00, 2),
('Triple', 350.00, 3),
('Quad', 450.00, 4),
```

```
INSERT INTO rooms (room_number, type_id, location_id, status) VALUES
(101, 1, 1, 'Available'),
(102, 2, 2, 'Occupied'),
(103, 3, 3, 'Available'),
(104, 4, 4, 'Occupied'),
(105, 5, 5, 'Available'),
```

```
INSERT INTO room_locations (college, gender_type) VALUES
('Alpha College', 'Male'),
('Beta College', 'Female'),
('Gamma College', 'Male'),
('Delta College', 'Female'),
('Epsilon College', 'Male'),
```

```
INSERT INTO reservations (reservation_id, student_id, room_id, start_date, end_date) VALUES
(1, 1, 1, '2025-12-01', '2025-12-10'),
(2, 2, 2, '2025-12-02', '2025-12-12'),
(3, 3, 3, '2025-12-03', '2025-12-15'),
(4, 4, 4, '2025-12-05', '2025-12-20'),
```

```
INSERT INTO payments (payment_id, reservation_id, amount, method) VALUES
(1, 1, 150.00, 'FPX'),
(2, 2, 250.00, 'CARD'),
(3, 3, 350.00, 'CASH'),
(4, 4, 450.00, 'FPX'),
```

1) Inserted sample data into all main tables:

- customers
- rooms
- reservations
- payments

2) Ensured foreign key order to avoid constraint errors.

3) Created realistic data for testing queries and system workflows.

Updating Existing Data (UPDATE Statements)

#Updating Student phone number

```
UPDATE students
```

```
SET phone = '0192345890'
```

```
WHERE student_id = 1;
```

- Updated certain values to correct data inconsistencies.
- Demonstrated how the system handles changes after records are created.
- Used WHERE clause to avoid updating unintended rows.

#updating room status

```
UPDATE rooms
```

```
SET status = 'Occupied'
```

```
WHERE room_id IN (101, 108);
```

Deleting Data (DELETE Statements)

```
-- Disable safe updates
```

```
SET SQL_SAFE_UPDATES = 0;
```

```
-- Delete reservations
```

```
DELETE FROM reservations
```

```
WHERE end_date < '2025-12-15';
```

```
-- Re-enable safe updates
```

```
SET SQL_SAFE_UPDATES = 1;
```

- Demonstrated deletion of outdated or invalid records.
- Carefully used conditions to avoid accidental full-table deletion.
- Tested system behavior when linked records are removed.

DQL / SELECT

FILTERING

This query finds reservations that start between 1 December and 31 December but have an end date earlier than 20 December.

```
#finding reservations between start date 1/12 - 31/12 but end date <20/12
SELECT *
FROM reservations
WHERE start_date BETWEEN '2025-12-01' AND '2025-12-31'
AND NOT end_date > '2025-12-20';
```

reservation_id	student_id	room_id	start_date	end_date	reserve_date
3	3	3	2025-12-03	2025-12-15	2025-11-24
4	4	4	2025-12-05	2025-12-20	2025-11-24
5	5	5	2025-12-06	2025-12-18	2025-11-24
6	6	6	2025-12-07	2025-12-17	2025-11-24
7	7	7	2025-12-08	2025-12-19	2025-11-24
NULL	NULL	NULL	NULL	NULL	NULL

LEFT OUTER JOIN

This query lists every room, including those without reservations. NULL appears for rooms with no matching reservation.

#c.LEFT OUTER - show all rooms, even if not reserved

```
SELECT
    rm.room_id,
    rm.room_number,
    r.reservation_id
FROM rooms rm
LEFT JOIN reservations r ON rm.room_id = r.room_id;
```

room_id	room_number	reservation_id
1	101	NULL
2	102	NULL
3	103	3
4	104	4
5	105	5
6	106	6
7	107	7
8	108	8
9	109	9
10	110	10

INDEXING

Optimizing Price Queries with B-TREE

```
# -- PART D - INDEXING & OPTIMIZATION
-- 1. BEFORE INDEXING
-- BTREE TEST (price > 200)
EXPLAIN ANALYZE
SELECT rm.room_number, rt.type, rt.price
FROM rooms rm
JOIN room_types rt ON rm.type_id = rt.type_id
WHERE rt.price > 200.00;

-- 2. CREATE INDEXES
-- BTREE INDEX on price column
CREATE INDEX idx_price_btree
ON room_types(price);
```

- Before Indexing:
 - Database performed a Full Table Scan (checking every row).
 - Query Cost: 4.75.
- Solution: Implemented idx_price_btree on the room_types table.
- After Indexing:
 - Enabled Range Access strategy.
 - Query Cost: Reduced to 2.42 (~50% efficiency boost).

Text Search Optimization (FULLTEXT)

```
-- FULLTEXT TEST -> college LIKE '%College%'
EXPLAIN ANALYZE
SELECT rl.college, rm.room_number
FROM room_locations rl
JOIN rooms rm ON rl.location_id = rm.location_id
WHERE rl.college LIKE '%College%';
```

```
-- 2.2 FULLTEXT INDEX on college column
CREATE FULLTEXT INDEX idx_college_fulltext
ON room_locations(college);
```

- Scenario: Searching for specific keywords (e.g., "College") in location names.
- Before Indexing:
- Used LIKE '%College%', forcing the database to read every character.
- Resulted in an inefficient Inner Hash Join.
- Solution: Implemented idx_college_fulltext and used MATCH...AGAINST.
- After Indexing:
- Utilized Natural Language Mode for tokenized searching.
- Query Cost: Drastically reduced to 0.70.

Validation of Results

```
-- SHOW created indexes  
  
SHOW INDEX FROM room_types;  
  
SHOW INDEX FROM room_locations;
```

- Verification: Executed SHOW INDEX to confirm the structure of B-TREE and FULLTEXT keys.
- Functional Testing:
 - Real-world queries now retrieve correct data
 - Eliminated bottlenecks in data retrieval.
- Final Outcome: The database is now fully optimized and scalable for future data growth.

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
▶	room_locations	0	PRIMARY	1	location_id	A	10	NULL	NULL		BTREE			YES	NULL
	room_locations	0	college	1	college	A	10	NULL	NULL		BTREE			YES	NULL
	room_locations	1	idx_college_fulltext	1	college	NULL	10	NULL	NULL		FULLTEXT			YES	NULL